

Step by Step Implementation on the project

1) Creating a Lambda Function:-

The screenshot shows the AWS Lambda 'Create function' page. At the top, there's a navigation bar with the AWS logo, 'Services', a search bar, and a keyboard shortcut '[Alt+S]'. Below the navigation bar, the breadcrumb trail reads 'Lambda > Functions > Create function'. The main heading is 'Create function' with an 'Info' link. A subtext says 'Choose one of the following options to create your function.' There are three selection boxes: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. The 'Author from scratch' box contains the text 'Start with a simple Hello World example.' Below these is the 'Basic information' section. It includes a 'Function name' field with the value 'LoanCharges_fn' and a note 'Enter a name that describes the purpose of your function.' Below the field is a note 'Use only letters, numbers, hyphens, or underscores with no spaces.' The 'Runtime' section has a dropdown menu set to 'Python 3.11' and a refresh button. Below that is the 'Architecture' section with two radio buttons: 'x86_64' (selected) and 'arm64'. At the bottom of the form is the 'Permissions' section with an 'Info' link. The footer of the console shows 'CloudShell' and 'Feedback' links.

aws Services Search [Alt+S]

Lambda > Functions > Create function

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

LoanCharges_fn

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.11

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.

☒ x86_64

☐ arm64

Permissions [Info](#)

CloudShell Feedback

2) Attach a Role with Dynamodb full access policy

The screenshot displays the AWS IAM console interface. The left sidebar shows the navigation menu with 'Identity and Access Management (IAM)' selected. The main content area is titled 'Summary' and shows the role's details. Below the summary, the 'Permissions' tab is active, displaying a list of attached policies.

Summary

Creation date June 12, 2024, 20:18 (UTC+05:30)	ARN arn:aws:iam::661355659378:role/service-role/serverless-api-demo
Last activity 2 hours ago	Maximum session duration 1 hour

Permissions | Trust relationships | Tags | Access Advisor | Revoke sessions

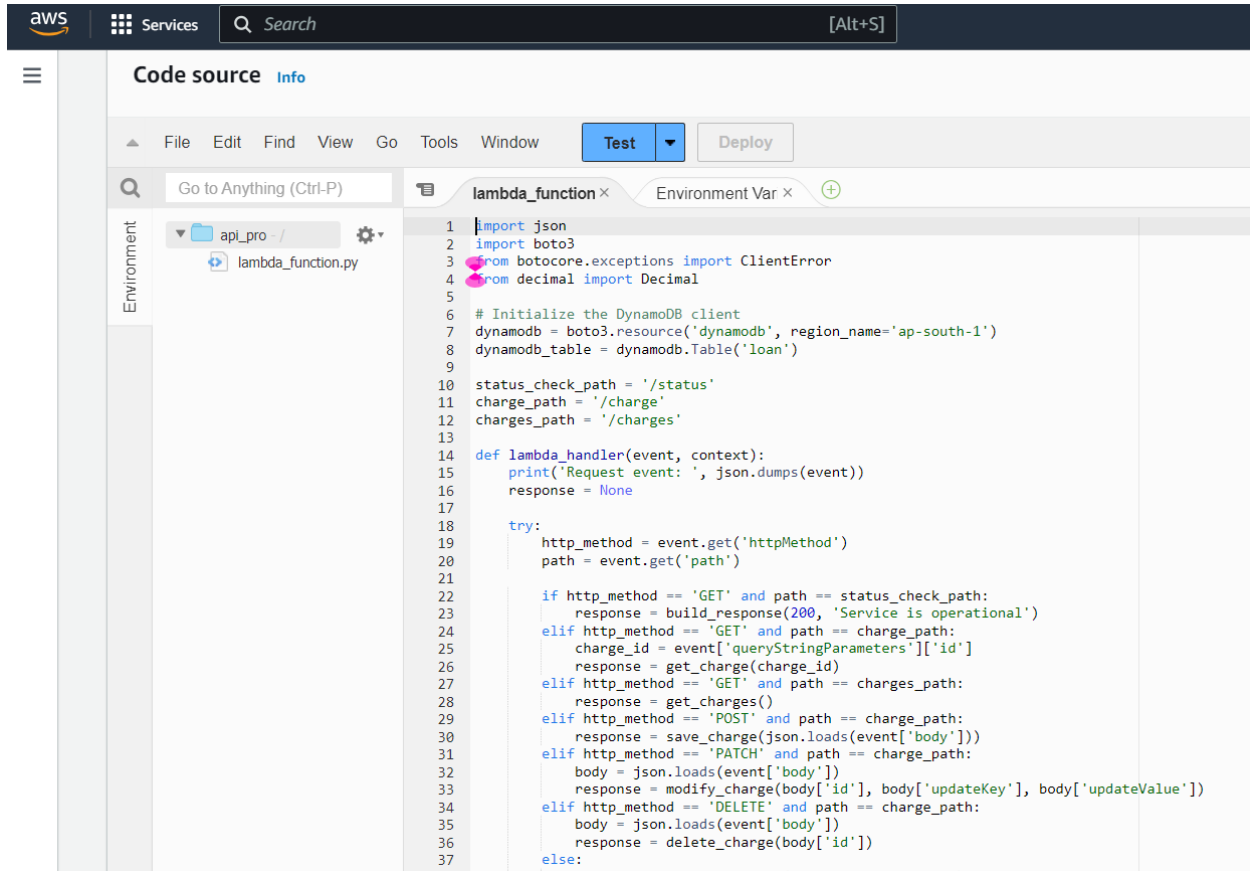
Permissions policies (3) Info

You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
☐ AmazonDynamoDBFullAccess	AWS managed	1
☐ AWSLambdaBasicExecutionRole-538a1c...	Customer managed	1
☐ CloudWatchLogsFullAccess	AWS managed	1

3) Deploy the Code on Amazon Lambda:-



The screenshot displays the AWS Lambda console's 'Code source' page. The top navigation bar includes the AWS logo, 'Services', a search bar, and a keyboard shortcut '[Alt+S]'. The main header shows 'Code source' with an 'Info' link. Below this is a menu bar with 'File', 'Edit', 'Find', 'View', 'Go', 'Tools', and 'Window', along with 'Test' and 'Deploy' buttons. A search bar 'Go to Anything (Ctrl-P)' is present. The left sidebar shows the 'Environment' tab with a file tree containing 'api_pro' and 'lambda_function.py'. The main editor area shows the code for 'lambda_function.py' with line numbers 1 through 37. The code imports 'json', 'boto3', and 'Decimal', initializes a DynamoDB client and table, and defines a 'lambda_handler' function that handles various HTTP methods (GET, POST, PATCH, DELETE) for different paths ('/status', '/charge', '/charges').

```
1 import json
2 import boto3
3 from botocore.exceptions import ClientError
4 from decimal import Decimal
5
6 # Initialize the DynamoDB client
7 dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
8 dynamodb_table = dynamodb.Table('loan')
9
10 status_check_path = '/status'
11 charge_path = '/charge'
12 charges_path = '/charges'
13
14 def lambda_handler(event, context):
15     print('Request event: ', json.dumps(event))
16     response = None
17
18     try:
19         http_method = event.get('httpMethod')
20         path = event.get('path')
21
22         if http_method == 'GET' and path == status_check_path:
23             response = build_response(200, 'Service is operational')
24         elif http_method == 'GET' and path == charge_path:
25             charge_id = event['queryStringParameters']['id']
26             response = get_charge(charge_id)
27         elif http_method == 'GET' and path == charges_path:
28             response = get_charges()
29         elif http_method == 'POST' and path == charge_path:
30             response = save_charge(json.loads(event['body']))
31         elif http_method == 'PATCH' and path == charge_path:
32             body = json.loads(event['body'])
33             response = modify_charge(body['id'], body['updateKey'], body['updateValue'])
34         elif http_method == 'DELETE' and path == charge_path:
35             body = json.loads(event['body'])
36             response = delete_charge(body['id'])
37     except:
```

4) Create a Table in DynamoDb:-

 Services [Alt+S]

 [DynamoDB](#) > [Tables](#) > Create table

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

String ▼

1 to 255 characters and case sensitive.

Sort key - *optional*
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

String ▼

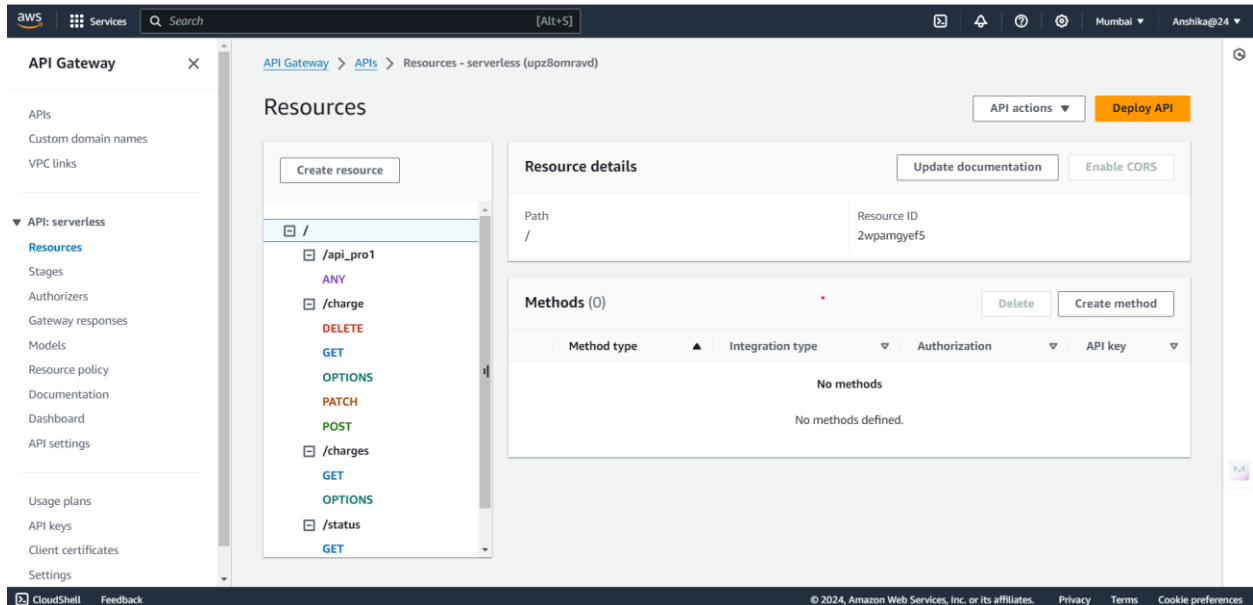
1 to 255 characters and case sensitive.

Table settings

 CloudShell [Feedback](#)

5) Create an API Gateway

- Make 3 resources in your API named status , charge and charges and enable Lambda Proxy Integration in it.
- Create GET method in status and charges.
- Create GET , POST , PATCH , DELETE in charge.



6) Deploy the API and copy the invoke url.

Invoke Url:- <https://upz8omravd.execute-api.ap-south-1.amazonaws.com/prod>

7) Open the invoke url in your browser to check the status of API's

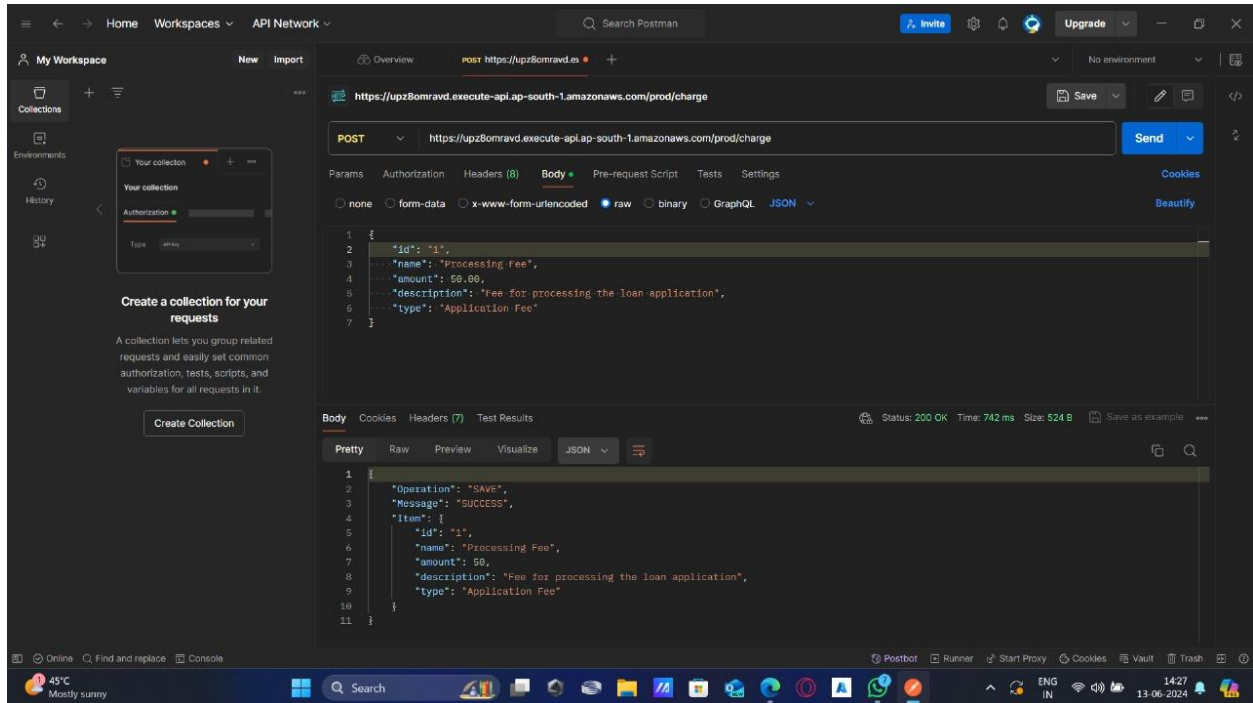
Invoke Url :- <https://upz8omravd.execute-api.ap-south-1.amazonaws.com/prod/status>

Testing

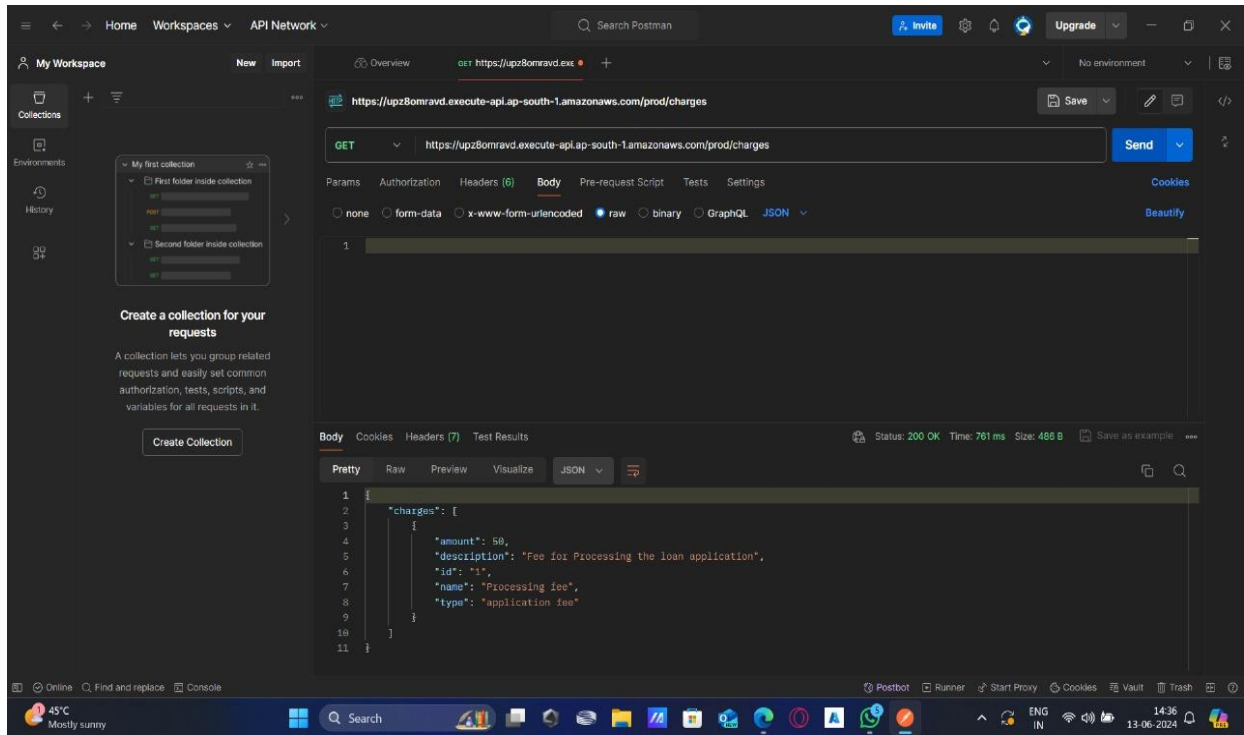
Now let's test our api's by sending request through POSTMAN

1) Create an entry in the table:-

Endpoint:- /charge , Method:- POST , Primary Key:- id

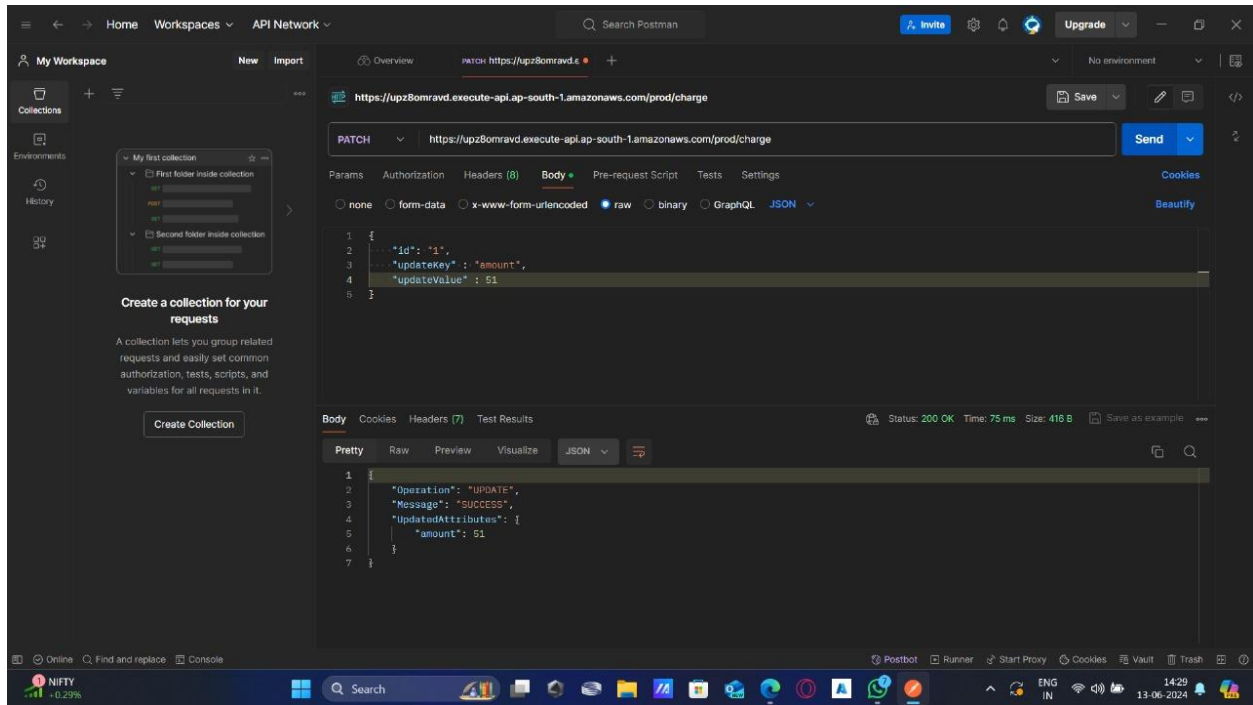


- 2) Retrieve all entries from the table:-
Endpoint:- /charges , Method: - GET



3) Update any value in the table:-

Endpoint:- /charge , Method:- PATCH



4) Delete an entry from the table:-

Endpoint:- /charge , Method:- DELETE

